



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

**«МИРЭА – Российский технологический университет»
РТУ МИРЭА**

Институт Информационных Технологий

Кафедра Вычислительной техники

**ОТЧЕТ О ВЫПОЛНЕНИИ ПРАКТИЧЕСКОЙ РАБОТЫ
№6**

«Реализация недетерминированного автомата»

по дисциплине

«Теория формальных языков»

Выполнил студент группы ИКБО-43-23

Кощев М. И.

Принял ассистент

Цынгалёв П.С.

Практическая работа
выполнена

«2» декабря 2024 г.

«Зачтено»

«__» _____ 2024 г.

Москва 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ЦЕЛЬ.....	4
2 ЗАДАНИЕ.....	5
2.1 Формулировка задания.....	5
2.2 Математическая модель.....	5
2.3 Тестирование программы.....	6
3 ЗАКЛЮЧЕНИЕ	8
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ.....	9
ПРИЛОЖЕНИЯ.....	10

ВВЕДЕНИЕ

Изучение теории формальных языков представляет собой значимый элемент образовательной программы по направлению программной инженерии. Эта область, являющаяся ключевой частью компьютерных наук и теоретической информатики, сосредотачивается на исследовании структуры и свойств формальных языков, а также на моделировании их обработки. В условиях современного общества, где возрастают объемы данных и потребность в эффективных методах их хранения, передачи и обработки, понимание основ формальных языков приобретает особую значимость.

Формальные языки находят применение в различных сферах: от описания синтаксиса языков программирования до разработки сетевых протоколов и создания компиляторов. С их помощью удастся формализовать взаимодействие как между человеком и компьютером, так и между компонентами программного обеспечения. В центре изучения теории формальных языков находятся такие понятия, как грамматики, автоматы и алгебраические структуры, которые позволяют описывать синтаксис и семантику языков.

В данном отчете мы рассмотрим реализацию недетерминированного автомата.

1 ЦЕЛЬ

Реализовать недетерминированный автомат согласно варианту.

2 ЗАДАНИЕ

2.1 Формулировка задания

На выбранном ЯП реализовать недетерминированный автомат согласно варианту.

Вариант: $v = (a|b) * \&(a|b)(a|b)$

2.2 Математическая модель

Создадим математическую модель для НКА (Недетерминированного Конечного Автомата). Для этого определим множество состояний автомата Q , содержащее состояния $q_0, q_1, \dots, q_n$, где q_0 — начальное состояние, а q_n — конечное состояние. Зададим множество входных символов Σ , которое включает все символы, используемые в регулярном выражении, а также ϵ для пустых переходов. Функция переходов δ описывает, как автомат переходит из одного состояния в другое на основании текущего символа. Начальное состояние автомата q_0 определяет, с какого состояния начинается обработка. Множество конечных состояний F включает в себя те состояния, в которых автомат может завершить работу.

Реализуем функцию, которая строит НКА на основе входного регулярного выражения. Она создаёт состояния автомата последовательно, начиная с начального состояния. Для каждого символа регулярного выражения создаётся переход между состояниями. Операции объединения ($|$), конкатенации ($.$) и замыкания ($*$) преобразуются в дополнительные состояния и переходы согласно их математическим определениям. Для объединения ($|$) создаются новые начальное и конечное состояния с ϵ -переходами к начальным состояниям двух объединяемых автоматов. Для конкатенации ($.$) два автомата соединяются ϵ -переходом от конечного состояния первого автомата к началу второго. Для замыкания ($*$) добавляются новые начальное и конечное состояния с ϵ -переходами, образующими циклы.

На вход подаётся строка, которая обрабатывается автоматом. Каждый символ строки проверяется на соответствие функции переходов δ . Если переход возможен, автомат переходит в новое состояние. Если на каком-то шаге переход невозможен, строка отклоняется. После обработки всех символов проверяется, находится ли автомат в одном из конечных состояний. Если находится, строка принимается, и выводится сообщение: «Строка принята». Если нет — «Строка не принята»

Графическое представление недетерминированного автомата продемонстрировано на рисунке 1.

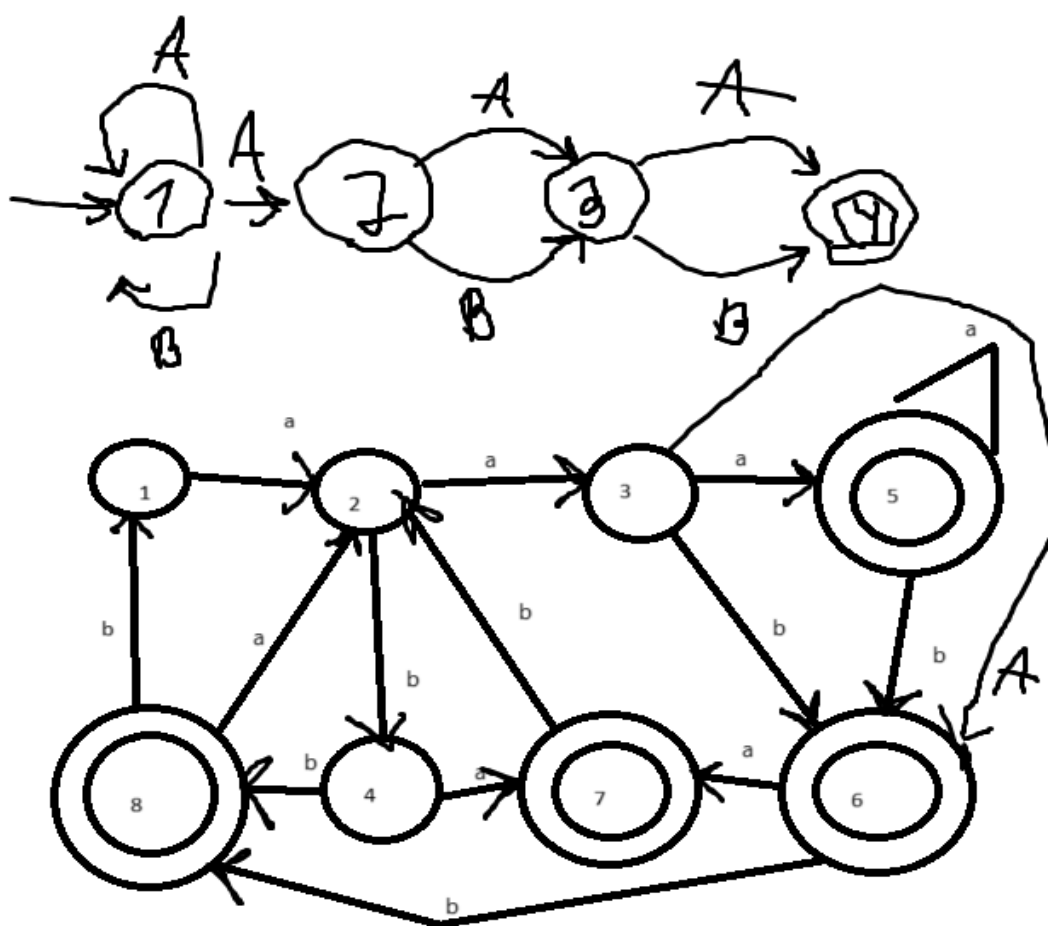


Рисунок 1 – Графическое представление недетерминированного конечного автомата

2.3 Тестирование программы

Проведём тестирование для данной программы на разных значениях. Результаты продемонстрируем на рисунке 2.

Введите регулярное выражение: $(a|b)^*\delta(a|b)(a|b)$

Сконструированный НКА:

Начальное состояние: 14

Конечное состояние: 21

Переходы:

0 --(a)→ 1
2 --(|)→ 3
4 --(b)→ 5
3 --()→ 4
1 --()→ 2
6 --()→ 0
5 --()→ 0
5 --()→ 7
6 --()→ 7
8 --(δ)→ 9
10 --(a)→ 11
12 --(b)→ 13
14 --()→ 10
14 --()→ 12
11 --()→ 15
13 --()→ 15
16 --(a)→ 17
18 --(b)→ 19
20 --()→ 16
20 --()→ 18
17 --()→ 21
19 --()→ 21
15 --()→ 20

Рисунок 2 - Тестирование программы

3 ЗАКЛЮЧЕНИЕ

В процессе выполнения практической работы были достигнуты следующие результаты:

Реализована функция создания недетерминированного конечного автомата;

Реализована функция обработки вводимой строки;

Проведено тестирование программы, и результаты представлены в отчете.

Таким образом, поставленная цель работы — реализация недетерминированного автомата — успешно достигнута.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1. Малявко А.А. Формальные языки и компиляторы: учебное пособие для вузов. - М., Изд-во Юрайт, 2019.
2. Свердлов С.З. Языки программирования и методы трансляции: учебное пособие для вузов - СПб., 2007, Id=65534.
3. Карпов Ю.Г. Теория и технология программирования. Основы построения трансляторов: учеб. пособие. - СПб.: БХВ-Петербург, 2005, Id=64347.

ПРИЛОЖЕНИЯ

Приложение А – код на языке Python

ПРИЛОЖЕНИЕ А

Код на языке Python

Листинг А.1 – main.py

```
def create_state(current_state):
    state = current_state
    current_state += 1
    return state, current_state

def add_transition(transitions, from_state, to_state, symbol):
    transitions.append((from_state, to_state, symbol))

def connect_nfa(stack, transitions):
    nfa2_start, nfa2_end = stack.pop()
    nfa1_start, nfa1_end = stack.pop()
    add_transition(transitions, nfa1_end, nfa2_start, None)
    stack.append((nfa1_start, nfa2_end))

def union_nfa(stack, transitions, current_state):
    nfa2_start, nfa2_end = stack.pop()
    nfa1_start, nfa1_end = stack.pop()
    new_start, current_state = create_state(current_state)
    new_end, current_state = create_state(current_state)
    add_transition(transitions, new_start, nfa1_start, None)
    add_transition(transitions, new_start, nfa2_start, None)
    add_transition(transitions, nfa1_end, new_end, None)
    add_transition(transitions, nfa2_end, new_end, None)
    stack.append((new_start, new_end))
    return current_state

def closure_nfa(stack, transitions, current_state):
    start, end = stack.pop()
    new_start, current_state = create_state(current_state)
    new_end, current_state = create_state(current_state)
    add_transition(transitions, new_start, start, None)
    add_transition(transitions, end, start, None)
    add_transition(transitions, end, new_end, None)
    add_transition(transitions, new_start, new_end, None)
    stack.append((new_start, new_end))
    return current_state
```

Продолжение листинга A.1

```
def process_operator(operators, stack, transitions, current_state):
    operator = operators.pop()
    if operator == '|':
        current_state = union_nfa(stack, transitions,
current_state)
    elif operator == '.':
        connect_nfa(stack, transitions)
    return current_state

def compile_nfa(pattern):
    stack = []
    operators = []
    current_state = 0
    transitions = []

    precedence = {'|': 1, '.': 2}

    i = 0
    while i < len(pattern):
        char = pattern[i]

        if char == '(':
            operators.append(char)

        elif char == ')':
            while operators and operators[-1] != '(':
                current_state = process_operator(operators, stack,
transitions, current_state)
            operators.pop()

        elif char == '*':
            current_state = closure_nfa(stack, transitions,
current_state)

        elif char in precedence:
            while (operators and operators[-1] != '(' and
                precedence[operators[-1]] >= precedence[char]):
                current_state = process_operator(operators, stack,
transitions, current_state)
            operators.append(char)

        else:
            start, current_state = create_state(current_state)
            end, current_state = create_state(current_state)
            add_transition(transitions, start, end, char)
            stack.append((start, end))
```

Продолжение листинга A.1

```
if i + 1 < len(pattern) and pattern[i + 1] not in '|*)':
    operators.append('.')

    i += 1

while operators:
    current_state = process_operator(operators, stack,
transitions, current_state)

    start_state, end_state = stack.pop()
    return start_state, end_state, transitions

def print_nfa(start_state, end_state, transitions):
    print(f"Начальное состояние: {start_state}")
    print(f"Конечное состояние: {end_state}")
    print("Переходы:")
    for (from_state, to_state, symbol) in transitions:
        symbol_str = symbol if symbol is not None else ''
        print(f"  {from_state} --({symbol_str})--> {to_state}")

if __name__ == "__main__":
    pattern = input("Введите регулярное выражение: ")
    start, end, transitions = compile_nfa(pattern)
    print("\nСконструированный НКА:")
    print_nfa(start, end, transitions)
```